

Challenge Write-Up Template

Challenge Information

Student Name	Daniella Marie A. Leonud
Section / Class	BSIT-3G
Challenge Name	Password Profiler
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 9, 2026
Flag Format Given	picoCTF{Aj_15901990}

1. Challenge Description

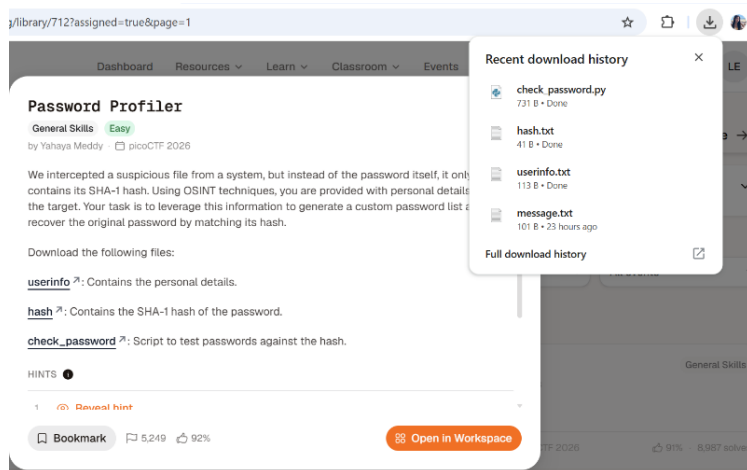
Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copy-paste the original prompt.

In this Password Profiler challenge, I was given information about a person and a SHA-1 hash. The goal was to use the person's details to create possible password combinations and then find which password matched the given hash. For this challenge, I was given a Python tool called CUPP (Common User Passwords Profiler). CUPP can create a custom wordlist by combining information such as a person's name, nickname, birthdate, and family names. After generating the wordlist, I needed to test the generated passwords against the SHA-1 hash provided by the challenge.

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

First, I read the challenge description and hint and downloaded all the files that were needed. After downloading them, I opened each file to understand what the challenge was asking me to do. The first file I checked was **userinfo.txt**. It contained personal information about the target, including her name, nickname, birthdate, and family names. I also opened **hash.txt**, which contained the SHA-1 hash **968c234904273dd57dc4be7e238c5ac200ceac5**. Lastly, I opened **check_password.py**. From reading the code, I understood that the script was supposed to read passwords from **passwords.txt**, convert each password into a hash, and compare it with the hash stored in hash.txt. I used **Notepad** and **Antigravity** to open and inspect the downloaded files.



3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

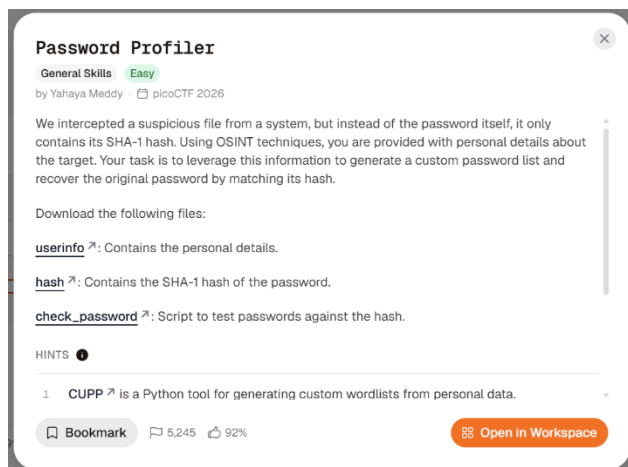
Tool / Command	What it was used for	Result / Output
python3 cupp.py -i	Launched CUPP in interactive mode to answer the target profiling questions.	Collected target info and generated alice.txt with 29,516 words.
cp alice.txt ../passwords.txt	Copied the generated wordlist to the main directory	Saved the wordlist file as passwords.txt.
cd ..	Moved up one folder level back to the main workspace directory.	Navigated back to terminal.
python3 check_password.py	Attempted to run the challenge's default checking script.	Returned error: No such file or directory.
python3 -c "import hashlib..."	Ran a one-line Python script to hash each word in passwords.txt and match it against the SHA-1 hash.	Found the matching password and printed picoCTF{Aj_15901990}.
CUPP (Common User Passwords Profiler)	The main profiling tool used to generate a custom wordlist based on personal details.	Created a dictionary of common password combinations.
Gemini AI	Provided step-by-step guidance, solved terminal errors, and generated the Python command to crack the hash.	Guided the entire challenge process to successfully get the flag.
Notepad	Used to view and check the contents of downloaded text files and scripts.	Displayed plain text files clearly.
Antigravity	Used to open, view, and inspect downloaded files and web resources.	Rendered and opened files for review.

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

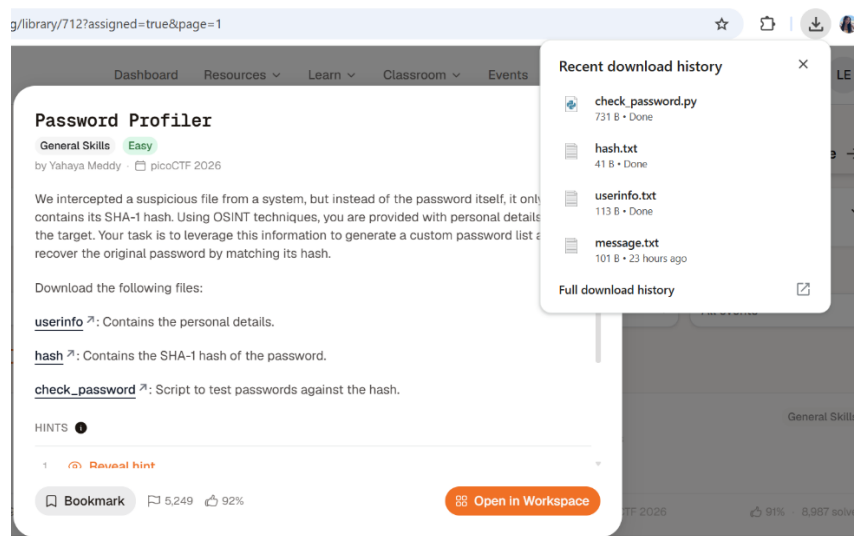
Step 1

First, I opened the Password Profiler challenge and read the description and hint carefully. After that, I downloaded all the files provided by the challenge because I needed to inspect them before starting the actual password-generation process.



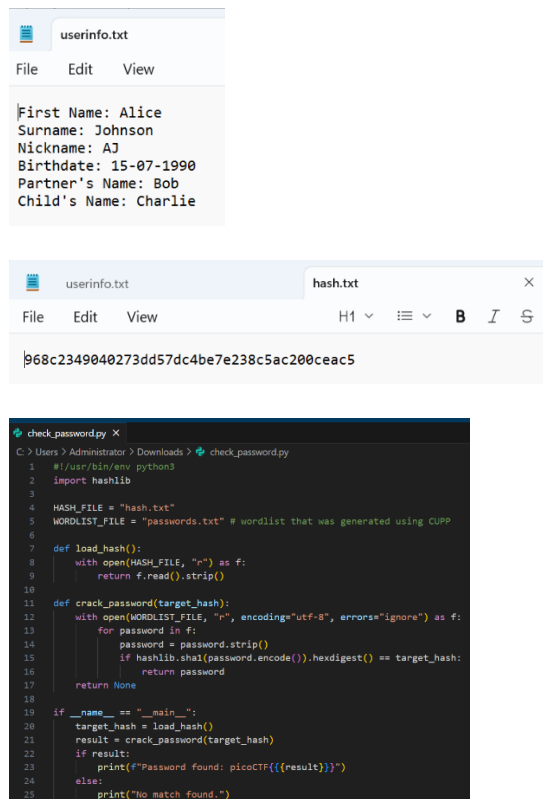
Step 2

After downloading the files, I opened each one to understand what information they contained. I used Notepad and Antigraity to view the files. **userinfo.txt**, **hash.txt**, **check_password.py**



Step 3

I started by opening and reading the downloaded files. In the **userinfo.txt** it showed personal details like her name, nickname, birthdate, and family names. In **hash.txt** it contained the target hash which is 968c234904273dd57dc4be7e238c5ac200ceac5. Lastly, the **check_password.py**, it is a Python code that reads passwords.txt, turns each word into a hash, and compares it to hash.txt. I used Notepad and AntigraVity to open these files.



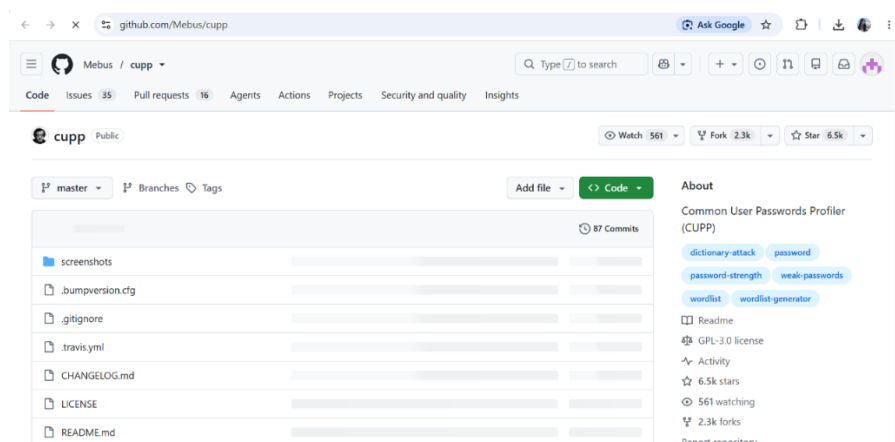
```
userinfo.txt
File Edit View
First Name: Alice
Surname: Johnson
Nickname: AJ
Birthdate: 15-07-1990
Partner's Name: Bob
Child's Name: Charlie

hash.txt
File Edit View H1 H2 H3 H4 H5 H6 H7 H8 H9 H0 B I S
968c234904273dd57dc4be7e238c5ac200ceac5

check_password.py X
C:\Users\Administrator\Downloads> check_password.py
1 #!/usr/bin/env python3
2 import hashlib
3
4 HASH_FILE = "hash.txt"
5 WORDLIST_FILE = "passwords.txt" # wordlist that was generated using CUPP
6
7 def load_hash():
8     with open(HASH_FILE, "r") as f:
9         return f.read().strip()
10
11 def crack_password(target_hash):
12     with open(WORDLIST_FILE, "r", encoding="utf-8", errors="ignore") as f:
13         for password in f:
14             password = password.strip()
15             if hashlib.sha1(password.encode()).hexdigest() == target_hash:
16                 return password
17     return None
18
19 if __name__ == "__main__":
20     target_hash = load_hash()
21     result = crack_password(target_hash)
22     if result:
23         print(f"Password found: picoCTF{{{result}}}")
24     else:
25         print("No match found.")
```

Step 4

I also opened the CUPP from the hint, but since I do not know how to use it yet, I relied on AI assistance and decided to use the terminal and some commands to finish this challenge.



Step 5

To create a custom list of passwords based on Alice's details, I needed the CUPP tool. I downloaded it directly into the terminal from GitHub by running. This created a new folder named `cupp` containing all the tool's files. Next, I entered the tool's folder and launched CUPP in interactive mode:

```
For more information and a beginner's guide, type less ~/README.txt.

=====

leonuddaniella-academy@webshell:~$ git clone https://github.com/Mebus/cupp.git
Cloning into 'cupp'...
remote: Enumerating objects: 266, done.
remote: Counting objects: 100% (130/130), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 266 (delta 86), reused 69 (delta 69), pack-reused 136 (from 1)
Receiving objects: 100% (266/266), 2.16 MiB | 1.80 MiB/s, done.
Resolving deltas: 100% (144/144), done.
leonuddaniella-academy@webshell:~$ cd cupp
```

Step 6

The program asked a series of questions about the target, and I typed in Alice's details. CUPP combined all these pieces of information into thousands of password variations and saved them in a file named `alice.txt`

```
leonuddaniella-academy@webshell:~/cupp$ python3 cupp.py -i

cupp.py!

# Common
# User
# Passwords
# Profiler

[ Muris Kurgas | j0rgan@remote-exploit.org ]
[ Mebus | https://github.com/Mebus/ ]

[+] Insert the information about the victim to make a dictionary
[+] If you don't know all the info, just hit enter when asked! ;)
```

```
> First Name: Alice
> Surname: Johnson
> Nickname: AJ
> Birthdate (DDMMYYYY): 15071990

> Partners) name: Bob
> Partners) nickname:
> Partners) birthdate (DDMMYYYY):

> Child's name: Charlie
> Child's nickname:
> Child's birthdate (DDMMYYYY):

> Pet's name:
> Company name:
```

Step 7

The program asked more questions about the target, and I just need to type y or n. If I want to skip it, then proceed with entering. The checking script (**check_password.py**) was located in the main directory and needed a wordlist named passwords.txt. I copied alice.txt to the parent folder and renamed it. Then, I moved back up to the main folder using (**cd ..**)

```
> Do you want to add some key words about the victim? Y/[N]: y
> Please enter the words, separated by comma. [i.e. hacker,juice,black], spaces will be removed:
> Do you want to add special chars at the end of words? Y/[N]: y
> Do you want to add some random numbers at the end of words? Y/[N]:y
> Leet mode? (i.e. leet = 1337) Y/[N]: y

[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[+] Saving dictionary to alice.txt, counting 29516 words.
> Hyperspeed Print? (Y/n) : n
[+] Now load your pistolero with alice.txt and shoot! Good luck!
leonuddaniella-academy@webshell:~/cupp$ cp alice.txt ../passwords.txt
leonuddaniella-academy@webshell:~/cupp$ cd ..
leonuddaniella-academy@webshell:~$ python3 check_password.py
python3: can't open file '/home/leonuddaniella-academy/check_password.py': [Errno 2] No such file or directory
```

Step 8

Since check_password.py was missing from the terminal directory, I used a one-line Python command instead. (**import hashlib**) Imports Python's built-in tool for working with cryptographic hashes. So when the command opened passwords.txt, I used hashlib to convert each word into a SHA-1 hash, and compared it to the target hash.

```
leonuddaniella-academy@webshell:~$ python3 check_password.py
python3: can't open file '/home/leonuddaniella-academy/check_password.py': [Errno 2] No such file or directory
leonuddaniella-academy@webshell:~$ python3 -c "import hashlib; [print('Password found:', f'picoCTF{{{line.strip()}}}') for line in open('passwords.txt', errors='ignore') if hashlib.sha1(line.strip().encode()).hexdigest() == '968c2349040273dd57dc4be7e238c5ac200ceac5']"
Password found: picoCTF{Aj_15901990}
Password found: picoCTF{Aj_15901990}
```

Step 9

The challenge is successful, and I got the correct flag.

Password Profiler

General Skills Easy

by Yahaya Meddy · picoCTF 2026

1

CUPP is a Python tool for generating custom wordlists from personal data.

FLAG

picoCTF{Aj_15901990}

Submit

Correct flag!

5. Flag

```
picoCTF{Aj_15901990}
```

6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

Well, this challenge taught me how weak passwords made from personal details can be easily guessed. So, tools like CUPP can automatically combine names and birthdates into thousands of possible passwords. It also showed how Python can quickly test those passwords against a hash to find the correct one. Personally, knowing this kind of tool is actually helpful because it helps us protect our digital profile.

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

CUPP Tool (<https://github.com/Mebus/cupp>)

Python Hashlib (<https://docs.python.org/3/library/hashlib.html>)

Gemini AI (<https://gemini.google.com/>)
